How the pipeline moved from a 6-tool logistics world to the confirmed benchmark contract: one BM25 `search` tool over a provided candidate passage set, answering MuSiQue-style multi-hop retrieval questions. This document walks through every change, why it was made, and how the rebuild was verified.

1 The change at a glance

	Before (legacy world)	After (new contract)
Tools	6: web_search, fetch_page, db_query, db_aggregate, calculator, search	1: search (BM25 ranked retriever over the given candidate set)
Domain	Logistics, products, warehouses	Wikipedia-style encyclopedic entities
Questions	Single-fact lookups + some multi-hop	MuSiQue multi-hop chains (2 / 3 / 4-hop)
Answer source	Internal database + external web	Passages supplied with each question
Headline metric	Exact / normalised match	Token-F1 (34% baseline → ~60% target)

In plain English: the whole task is now — read a nested multi-hop question, then use the single search tool over the passages you are given, retrieving the right passage one hop at a time until you can compose the answer. The calculator, database, web search and page-fetch tools are gone. This matches the public MuSiQue benchmark the judge uses.

2 What changed in each file

2.1 world.py — the encyclopedic world

Replaced the logistics domain with a seeded, deterministic encyclopedic universe. Each entity gets a Wikipedia-style passage (title + text + facts + links). The links are what create multi-hop questions: a person passage links to a company, the company to a city, the city to a country. Every episode builds its own world from a seed, so tools are executable and stateful without leaking between tasks.

Entity kinds in the world

- Countries — name, capital, region, population, official language (6)
- Geographic features — mountain ranges, rivers, bays, coastlines (5 per seed)
- Cities — each country's capital, with population and founding year
- Organisations — name, industry, HQ city, founding year (8 per seed)
- People — name, profession, birth city, employer (10 per seed)
- Works — novel / film / symphony, publishing year, author (3 per seed)

2.2 builtin.py — the single BM25 search tool

Replaced the six tools with exactly one: search, a genuine BM25 ranker (TF × IDF × length normalisation) over the episode's candidate passages. Title matches get a 1.6× boost because title words are strong entity signals.

- Empty query → a readable error telling the model to add search terms
- No matches → an empty result set (legitimate; the model must rephrase)
- Matches → full passage texts, supplying the next hop's query & the answer

2.3 schema.py — the task families

Family	What it tests	Tier
musique_2hop	A 2-link chain; one search resolves it	2
musique_3hop	A 3-link chain; two dependent searches	3
musique_4hop	A 4-link chain; three dependent searches	4
unanswerable	Info genuinely absent; the model must abstain	1
no_tool	Answerable from the model's own knowledge; a tool call is wrong	0

Old families (compute, db/doc multi-hop, recovery, action) were removed. The recovery family was dropped because it could not be made unambiguously verifiable (see §3).

2.4 generator.py — the MuSiQue generator

The heart of the rebuild. It mints multi-hop retrieval questions programmatically from the encyclopedic world, with no hand-labelling. Every task carries a natural nested prompt, a gold answer computed from the same world the tool reads, and an oracle_plan — the reference sequence of search calls (one per hop).

A real 3-hop question

```
What is the official language of the country that contains the city
where the company that employs the person who created Glass Rivers
is headquartered?
```

A 'route' is a list of relation steps; each follows a fact that names the NEXT entity. The route person → org → city → country builds a question whose answer needs four passages read in sequence. A 4-hop question therefore needs three dependent searches — you cannot write the second query until the first passage has been read. This dependency is exactly what forces (and teaches) sequential retrieval.

Example routes

Hops	Relation steps (the chain)	Leaf / answer
2-hop	feature → country → city	city population or founding year
2-hop	person → org → city	a city attribute
3-hop	feature → country → city → country	a country attribute
3-hop	work → person → org → city	a city attribute
4-hop	feature → country → city → country → city	a city attribute
4-hop	work → person → org → city → country	a country attribute

Data mix & the no_tool bank

Family	Share
musique_2hop	22%
musique_3hop	24%
musique_4hop	18%
unanswerable	11%
no_tool	25%

The mix weights the set toward multi-hop work (where score is won) while keeping the negative families as cheap insurance against the two default failure modes. The no_tool bank was rebuilt from scratch (~400 generic items — real capitals, chemical symbols, maths, spelling, sorting). Crucially, none of that information is present in the world, so a tool call cannot help.

3 What was removed, and why

The recovery family

The old recovery family generated tasks where a deliberately garbled first query returned nothing, forcing the model to rephrase. We removed it because it could not be made unambiguously verifiable: distinctive entity names almost always match the BM25 index, so an empty first query was essentially impossible to arrange reliably. Trying to force one with near-miss or ambiguous entities added complexity faster than it added signal. Self-correction from empty retrieval is still a modelled skill, but there is no dedicated training family for it.

The calculator and database tools

Removed along with the whole logistics world. Under the new contract there is no arithmetic tool, no database, and no external web — only BM25 retrieval over the provided passages. Simple arithmetic in no_tool-style questions is answered from the model's own knowledge.

4 A real bug found and fixed during the rebuild

While re-running the test suite we caught a genuine determinism bug. In gen_musique the code originally did:

```
routes = _ROUTES[hops]
rng.shuffle(routes)      # BUG: shuffles the GLOBAL _ROUTES list in place
```

Because _ROUTES is a module-level dictionary of lists, shuffling the returned list mutated the shared global in place. Two calls to generate(12000) with identical seeds could therefore pick different routes and produce different data — silently breaking reproducibility of the training set. The fix copies the list before shuffling:

```
routes = list(_ROUTES[hops])   # copy: shuffle mutates in place
rng.shuffle(routes)
```

Verified: two identical generate(40) calls now produce exactly 0 differing records.

5 Verification — the accuracy of the rebuild

Every number below was produced by running the actual code on this machine (CPU-only), not estimated. The honest caveat: these measure oracle / verifier correctness and pipeline integrity, not a model score — no real model has been trained or evaluated yet.

Check	Result
Oracle dev-eval — success / final / select / args / necessity	1.0 / 1.0 / 1.0 / 1.0 / 1.0
Full 12k dataset — oracle success	12000 / 12000
tests/test_pipeline.py	ALL PASS (26 checks)
tests/test_fix2.py	ALL PASS (23 checks)
tests/test_parser.py	ALL PASS (0 failures)
Determinism — two identical generate(40) runs	0 differing records
Downstream: raw → reject → SFT export	12000 → 2000 kept, balanced 400 / family
Tool calls in the exported SFT set	100% search (4000 calls)
Legacy-world residue in new dataset	0 (no web_search / db_query / calculator / Titan Loom)

6 Dataset regenerated

- Generate — 12,000 tasks from seeds 0..11999 using the new mix
- Oracle replay — every task through the oracle backend; 12,000 / 12,000 succeeded
- Rejection filter — 12,000 → 2,000 kept, re-balanced to 400 per family
- SFT export — 2,000 canonical records written, 0 skipped; all tool calls are search

Artifact	Contents
artifacts/raw_oracle.jsonl	The new 12k single-BM25 dataset (regenerated)
artifacts/raw_oracle.6tool.bak.jsonl	Backup of the old 6-tool dataset (kept for reference)

7 What's still open

- Token-F1 calibration — the judge scores by token-F1; our verifier uses exact / normalised match. Calibrating the two on the benchmark's own 54 examples is the highest-value next step.
- Harness wire format — how the organiser exposes the candidate set to the model. The last unknown for adapter.py.
- 3 / 4-hop phrasing variety — occasional template-y wording in the longest chains.

— end of report —